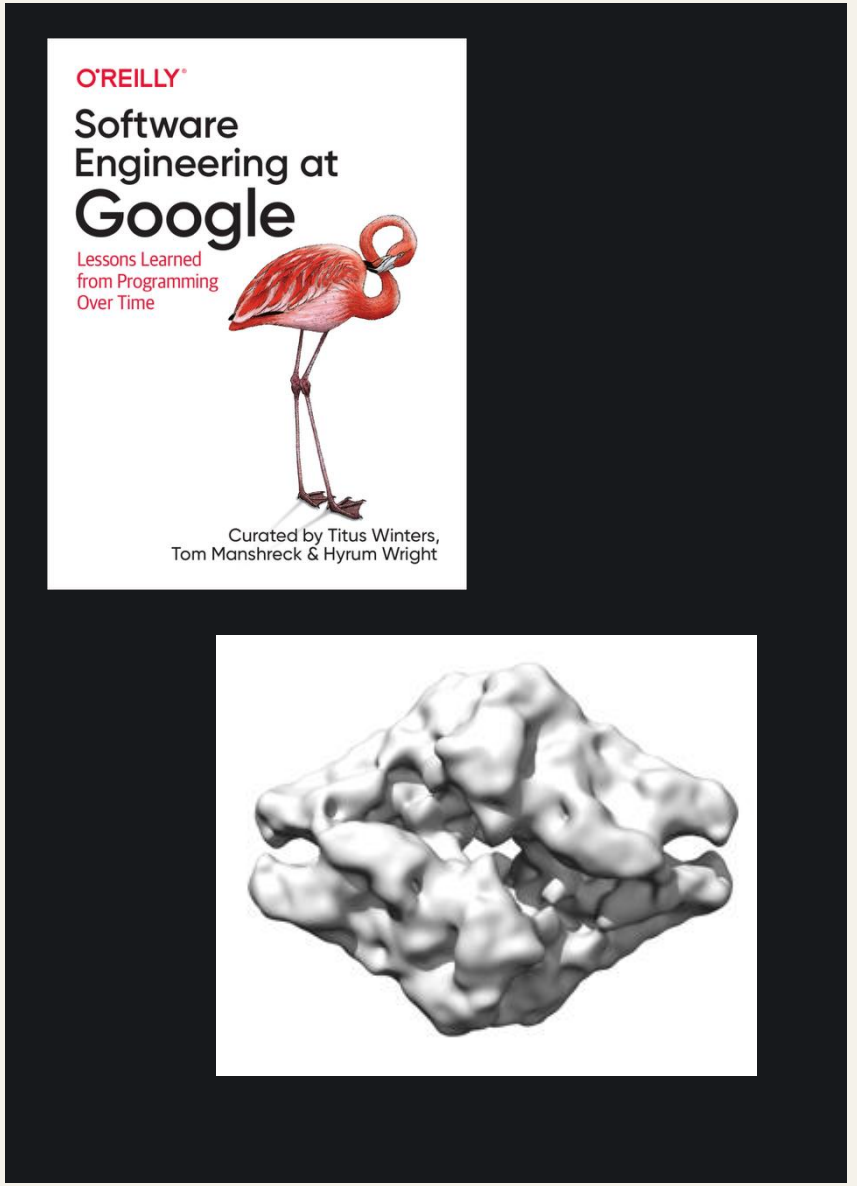
Testing SIMPLE

How years of cryo-EM development shaped a maintainable testing system

Fast unit gates, numerical truth oracles, simulated workflows, platform validation

A practical talk grounded in the SIMPLE repository



Before the unified suite: testing was diffuse

THE REPOSITORY SNAPSHOT

51 test_* directories

56 Fortran test programs

test_install

generated data and launched installed test executables

WHAT THAT FELT LIKE

Each test_* directory became its own standalone executable linked to SIMPLE.

The playground held prototypes, numerical experiments, and new algorithm ideas.

A regression test, smoke test, benchmark, demonstration, and method experiment could look almost identical.

Running successfully was often easier to see than a precise assertion with a failing exit status.

Our work was not merely adding tests. It was turning scattered programs into a test system that supports every new method.

Four testing questions for SIMPLE



TESTING OVERVIEW

Why SIMPLE needs automation at scientific-software scale



UNIT TESTING

How nearby testers and clear assertions keep changes maintainable



TEST DOUBLES

How phantoms, fixtures, and planted truths replace unavailable reality



LARGER TESTING

How worker-backed pipelines and platform runs restore fidelity

The organizing question: what evidence makes a numerical or workflow result trustworthy?

SIMPLE testing protects scientific change

A change can compile and run while still shifting poses, maps, metadata, or convergence behavior.

Numerical regressions can look plausible.

Parallel and platform differences can hide until deployment.

A workflow can complete without recovering the planted structure.

A file-lifecycle bug can corrupt later stages instead of failing immediately.

CURRENT BASELINE

42

CTest entries in the fixed process budget

15 fast area suites

19 high-level gates

SIMPLE implements the write, run, react loop

WRITE

A simple_<thing>_tester.f90 procedure asserts behavior next to the code it tests.

RUN

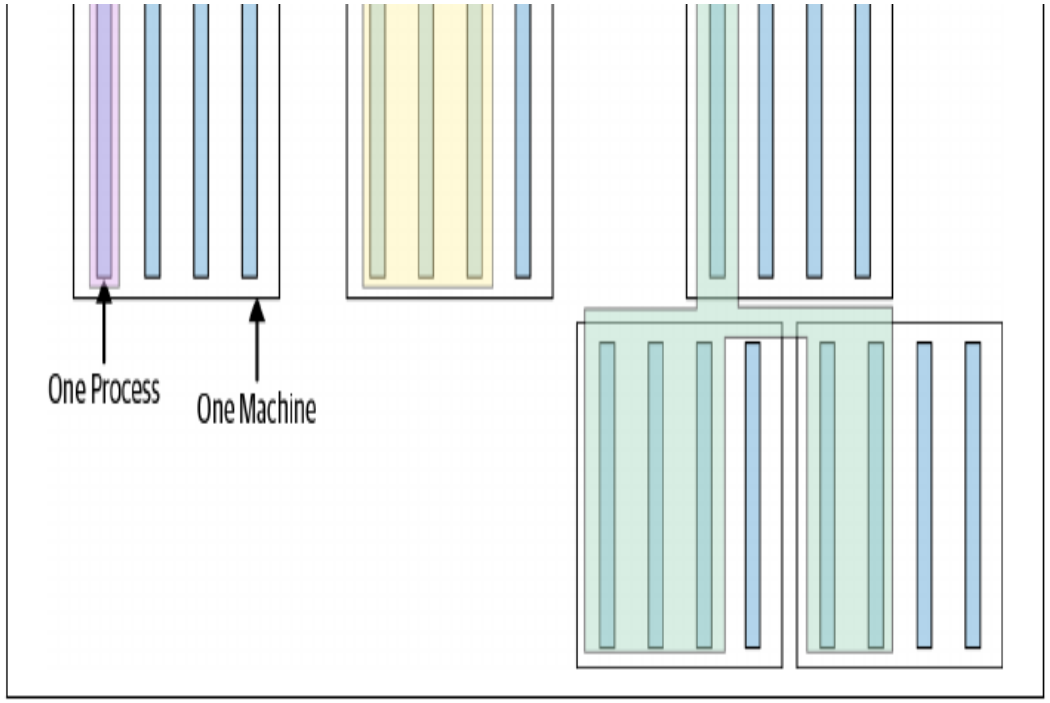
simple_test_exec routes every test through one executable, while compile scripts run the fast gate.

REACT

A failed entry returns a failing exit status and blocks installation until the defect is fixed.

A program that cannot fail through an assertion is not a test.

Test size and scope map cleanly onto SIMPLE



SIMPLE EXAMPLES

Small

In-process, one-thread unit area tests with generated data

Medium

Library suites using realistic boxes, numerical solvers, and longer generated fixtures

Large

Worker-backed pipelines, distributed reconstruction, and platform capabilities

Scope stays separate: a small test can verify a broad mathematical property, while a large test can focus on one workflow boundary.

SIMPLE's portfolio is intentionally tiered

Tier	Entries	When it runs	What it protects
Fast	15	Every normal compile before install	Hermetic unit behavior, metadata, numerics, and stage rules
Library	7	Nightly or focused development	Longer generated-data and numerical reconstruction checks
High level	19	Explicit CTest selection	Simulated workflows, commanders, workers, and scientific outcomes
Platform	1 baseline	Manual or capability-gated	Forking plus optional coarrays and accelerator behavior

The fixed CTest budget treats each entry as an isolation unit. New checks usually belong inside an existing suite.

GitHub Actions gives each test tier its own cadence

UNIT / FAST

MINUTES

SIMPLE push gate

A clean Ubuntu build runs the fast gate

Changes to source, build logic, or scripts trigger compile_clean.sh. A newer push cancels an older run.

MEDIUM

HOURS

SIMPLE nightly matrix

Linux and macOS run library and workflow groups

The groups finish independently and upload JUnit results, logs, reports, and metrics.

LARGE / REAL DATA

UP TO 24 H

SIMPLE_data_testing

Self-hosted runners exercise full cryo-EM datasets

Long campaigns preserve artifacts and publish reports that remain inspectable after the runner finishes.

Cadence follows cost. Fast evidence protects each change, while real-data evidence protects scientific behavior.

SIMPLE Actions protect source changes and nightly integration

PUSH GATE

Build SIMPLE on push

A clean Ubuntu checkout catches missing files, compiler differences, and fast-gate failures within minutes.

```
on: push to master
paths: src, production, cmake, scripts
runs-on: ubuntu-26.04
timeout: 30 minutes
run: ./compile_clean.sh
```

NIGHTLY MATRIX

Build SIMPLE & Run Tests

Hosted Linux and macOS runners split the suite by cost, then keep every group reporting even when another group fails.

```
schedule: every night
os: Ubuntu 26.04 and macOS
groups: library, workflows, rest, multistate
run: ctest by label and regular expression
artifacts: JUnit, logs, reports, metrics
```

The push workflow localizes ordinary regressions. The nightly matrix restores platform and process fidelity.

SIMPLE_data_testing exercises full workflows on real data

MAIN DATASET WORKFLOW

Run & Display SIMPLE Data Sets

Daily on a self-hosted Linux desktop or FRCE HPC runner

BETAGAL	APOF
beta-galactosidase	apoferritin
PROTEASOME	NANOX
proteasome	nanocrystal data

SPECIALIZED LONG CAMPAIGNS

100 streptavidin restarts

Measures stability across independent reconstructions.

Five controlled pose-error cases

Compares every reconstructed map against ground truth.

Beta-gal pose continuation

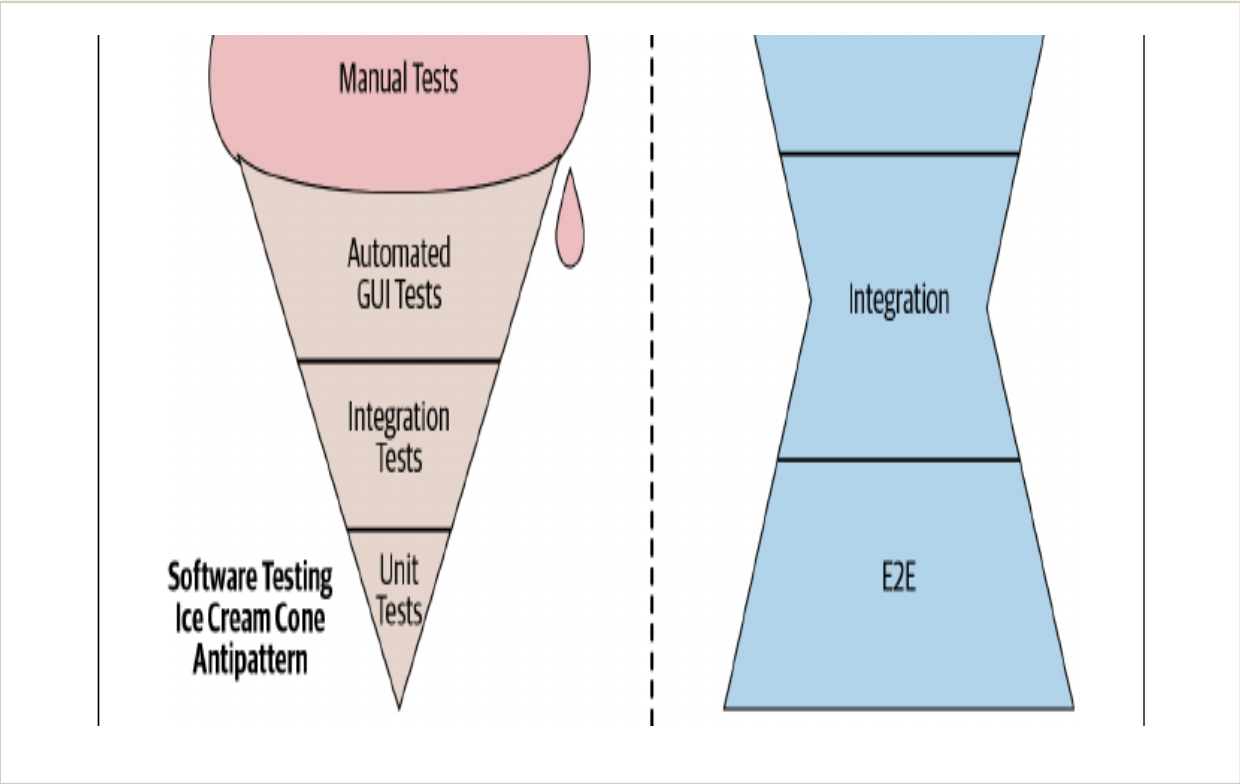
Tracks comparison results and successful run history.

Top-k comparison on three datasets

Runs a campaign, uploads evidence, and publishes a report.

Each campaign keeps logs and artifacts, then publishes a human-readable report through GitHub Pages.

Slow or toothless tests erode trust



THREE SIMPLE-SPECIFIC RISKS

No oracle

The workflow completes but no assertion judges the scientific result.

Uncontrolled randomness

A moving seed makes failures hard to reproduce or profile.

Only end-to-end coverage

A long pipeline reveals a defect but cannot localize it.

Maintainable SIMPLE tests live with their subject

PLACEMENT

simple_<thing>_tester.f90 sits next to the owning code

REGISTRATION

The tester becomes a sub-suite of its area suite

Why this matters

- Ownership is obvious during review.
- A focused selector runs only the touched area.
- Refactors update code and tests together.
- The suite avoids standalone scratch programs and one-off executables.

One executable keeps lifecycle, seed handling, reporting, and failure behavior consistent.

Unit testing: four connected concerns

PURPOSE AND SCOPE

A unit test covers a narrow public behavior

Most are also small and fast, but scope and execution size answer different questions.

MAINTAINABILITY

A harmless refactor leaves the test unchanged

Stable contracts prevent brittle failures and keep the suite useful as SIMPLE evolves.

CLEAR TESTS

The test body makes its behavioral claim visible

The name, relevant setup, observable result, and failure message tell one coherent story.

SHARED TEST CODE

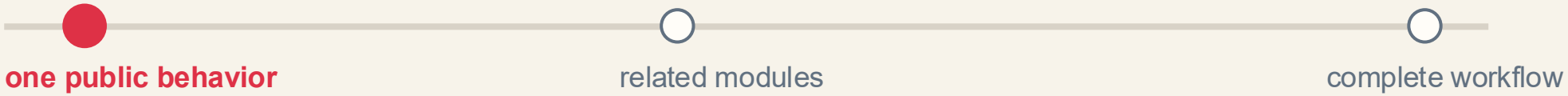
Reuse mechanics without hiding evidence

Focused helpers and common infrastructure support tests while the scientific claim stays local.

Unit scope and test size answer different questions

SCOPE

How much code and behavior does the test intend to validate?



SIZE

Which resources and execution constraints does the test use?



unit_stream has narrow scope, although some tests use local file fixtures

Existing tests should change only with behavior

PRODUCTION CHANGE	EXPECTED TEST RESPONSE
Pure refactor	Keep the existing behavior tests unchanged
New feature	Keep existing tests and add coverage for the new behavior
Bug fix	Add a regression test for the missing case
Behavior change	**Update the tests that define the changed contract**

Many failures after a refactor signal tests coupled to implementation details.

The owning layer defines the test boundary

UI / EXEC

Program metadata, defaults, visibility, and routing

COMMANDER / STRATEGY

Workflow orchestration, lifecycle, and project ownership

DOMAIN MODULE

Numerical algorithm, state transition, or data transformation

Stable test contract

Assert what the owning layer promises to callers.

Avoid checking private call order, temporary files, or intermediate representations unless they are part of the actual contract.

The same rule keeps orchestration in controllers and numerical truth in domain modules.

Clear tests answer five reader questions

BEHAVIOR	What contract does the test name promise?
SETUP	Which initial facts determine the expected result?
ACTION	Which public operation exercises that behavior?
RESULT	Which observable state proves the contract?
FAILURE	What expected and actual values will the report show?

A reader can verify the test without opening a helper or reconstructing its intent.

One unit test should describe one behavior

test_restart_removes_term_stream

The name states the behavior that will appear in a failure report.

GIVEN

A previous output folder

A stale TERM_STREAM marker remains from the earlier run.

WHEN

The p07 stage starts

The test constructs the real stage with that restart output.

THEN

The stage can continue

Restart is detected, the stale marker disappears, and finished() stays false.

The failure report names one behavioral guarantee.

Complete and concise tests keep the contract visible

```
call simple_mkdir('previous')
call simple_touch(TERM_STREAM)

call make_test_stage(stage, cline)

call assert_true(stage%l_restart, claim)
call assert_false(file_exists(TERM_STREAM), claim)
call assert_false(stage%finished(), claim)
```

The test body shows the contract from setup through verification.



COMPLETE

The body shows the initial state, the action, and every expected outcome.



CONCISE

Helpers may hide directory setup, default parameters, reporting, and cleanup.

A reader understands the claim without tracing helper code.

Simple oracles reveal mistakes at a glance

FILES ARRIVE OUT OF ORDER

00002.simple

00001.simple

The fixture states the disorder explicitly.

PRODUCTION ACTION

call stage%watch_sets()

The real watcher orders and records the exports.

EXPECTED STATE

first **00001.simple**

second **00002.simple**

second watch **still 2 records**

The expected order is written directly.

Reusing the watcher's sorting logic would weaken the oracle.

Assertions define verification. Failures report violated claims

call assert_real(expected, actual, tolerance, claim)

The assertion carries the expected value, observed result, allowed tolerance, and explanation.

ASSERTION

The executable claim

The test declares what must be true after the action.

ASSERTION FAILURE

The observed signal

The condition evaluates false, so the test reports the violated claim.

DEFECT

The cause to investigate

Broken behavior or a flawed test assumption produced the signal.

Runtime error or exception	The test may stop before it checks the scientific claim. An unexpected exception can fail a test, but it does not replace an oracle.

A SIMPLE assertion names the violated scientific claim

```
call assert_true(matched_by_truth(1) >= TRUTH_CORR_MIN, &
  tag//': a recovered state correlates with planted truth 1')

call assert_true(matched_by_truth(2) >= TRUTH_CORR_MIN, &
  tag//': a different recovered state correlates with planted truth 2')
```

Each assertion names one scientific claim

A failure reports which planted truth lost its distinct recovered state.

The oracle is label-invariant

The test accepts either state numbering, then checks one recovered state for each truth.

Shared test code should remove noise, not evidence



SHARED VALUES

Build the value each test needs

Keep the fields that determine the result in the test body. Defaults may supply irrelevant details.



SHARED SETUP

Reuse fixture mechanics

Common setup may create the stage and working directory. A test restates values central to its claim.



HELPERS

Give each helper one focused purpose

A helper may construct data or assert one conceptual fact. General validation hides the behavior under test.



TEST INFRASTRUCTURE

Treat shared infrastructure as a product

simple_test_utils owns assertion reporting and reproducible fixture mechanics. Its callers keep claims local.

SIMPLE shares test plumbing, not hidden intent

SHARE

simple_test_utils, fixed-seed setup, report formatting, fixture cleanup

KEEP VISIBLE

The planted truth, expected value, tolerance, and behavior being judged

Good helper

assert_real(expected, actual, tolerance, claim)

Risky helper

run_the_workflow_and_check_everything()

Test infrastructure behaves like production code: it needs ownership, its own tests, and a stable interface.

Reproducibility is part of SIMPLE's test contract

ONE FIXED ROOT SEED

simple_test_exec fixes the process seed before any commander or worker starts.

CONTROLLED VARIATION

seed_offset and test=profile explore multiple seeds without losing reproducibility.

Identity and profile answer different questions

IDENTITY	Do declared outputs match across two runs?
PROFILE	How robust is the scientific result across seeds and thread counts?

A stable seed makes a failure replayable. A profile prevents one lucky seed from becoming the specification.

Most SIMPLE simulations are synthetic-fixture tests

SYNTHETIC FIXTURE

A known cryo-EM world

Truth maps generate particles, poses, shifts, or movies. The expected scientific state is recorded before the test runs.

REAL SIMPLE CODE

The production path runs

Reconstruction, state assignment, picking, or stream stages execute without replacing the numerical implementation.

TRUTH ORACLE

The result faces independent evidence

Recovered maps, poses, particles, or states are compared with the planted truth rather than with call order.

SIMULATION-BASED TEST

Synthetic input replaces unavailable reality. The real system produces the state being judged.

MOCK OR INTERACTION TEST

A collaborator is replaced. The test verifies calls, arguments, or call order instead of the recovered scientific state.

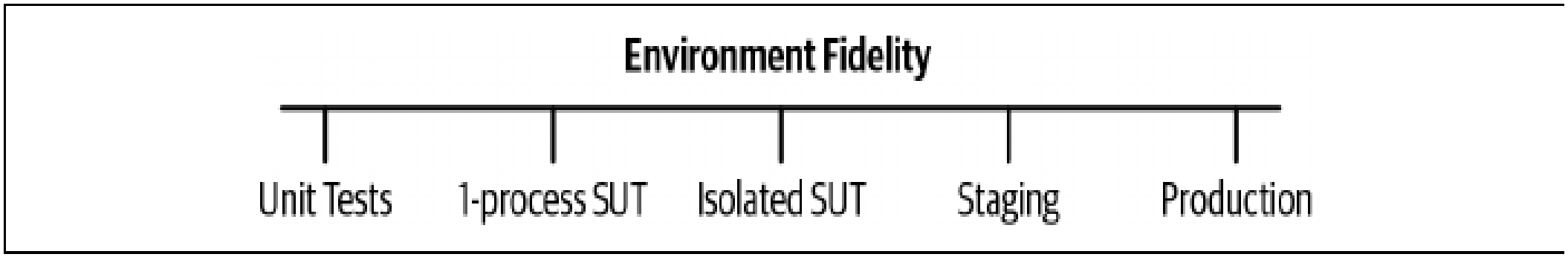
Expected values must come from independent evidence

CLOSED FORM	Analytic objective, transfer function, symmetry, or known physical limit
BRUTE FORCE	A small exhaustive calculation that the optimized algorithm must match
INDEPENDENT EMULATION	A separate implementation of the behavior at a manageable size
PLANTED TRUTH	Generated poses, maps, species, states, or particle coordinates

A test that calculates its expected value with the same implementation can reproduce the same bug and still pass.

Larger SIMPLE tests restore pipeline fidelity

A unit test can prove a Fourier operator or state transition. It cannot prove that a movie becomes the correct 3D structure through the full workflow.



SIMPLE unit and library tests

Worker-backed workflows and real platform runs

More fidelity catches configuration, lifecycle, scheduling, and integration defects, but raises runtime and diagnosis cost.

A SIMPLE workflow test has four explicit parts

SUT

A production stage, commander chain, or full workflow

DATA

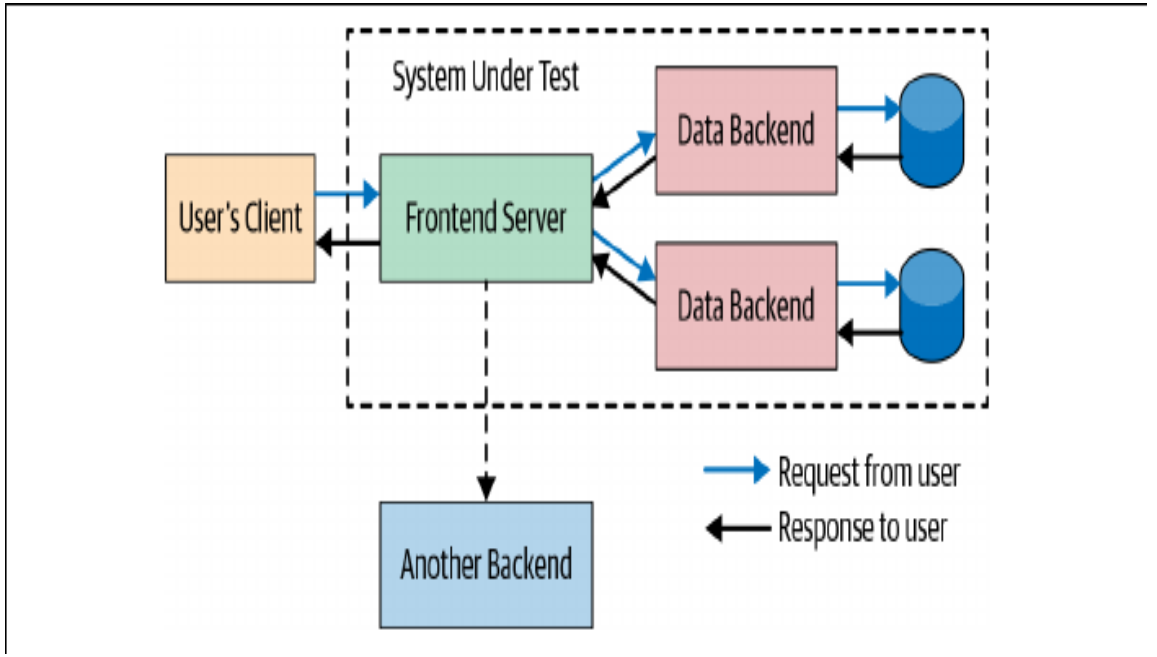
Seeded movies or particles plus planted truth

ACTION

Real SIMPLE commands, workers, and project transitions

VERIFICATION

Assertions against truth, identity, files, and lifecycle



For SIMPLE, the SUT boundary must say which stages, processes, and files belong to the claim.

The test plan starts from the risk and selects the smallest SUT that can expose it.

Three stream test tiers expand the system under test

TEST TIER	WHAT RUNS	QUESTION ANSWERED
unit_stream FAST GATE	**Stage steps, lifecycle, watchers, and pipes**	Does each stage behave correctly in isolation?
lib_stream NIGHTLY LIBRARY	**Real stream operations on generated fixtures**	Do grouping, picking, extraction, and docking match the fixture truth?
stream_chains HIGH-LEVEL	**Worker-backed stage chains against planted 3D truth**	Does the connected pipeline recover both planted states?

Each tier expands the SUT while keeping an independent oracle.

Case study: stream chains now prove scientific recovery



Old evidence

The chain completed and produced maps.

Current oracle

Both recovered states must match two distinct planted truths.

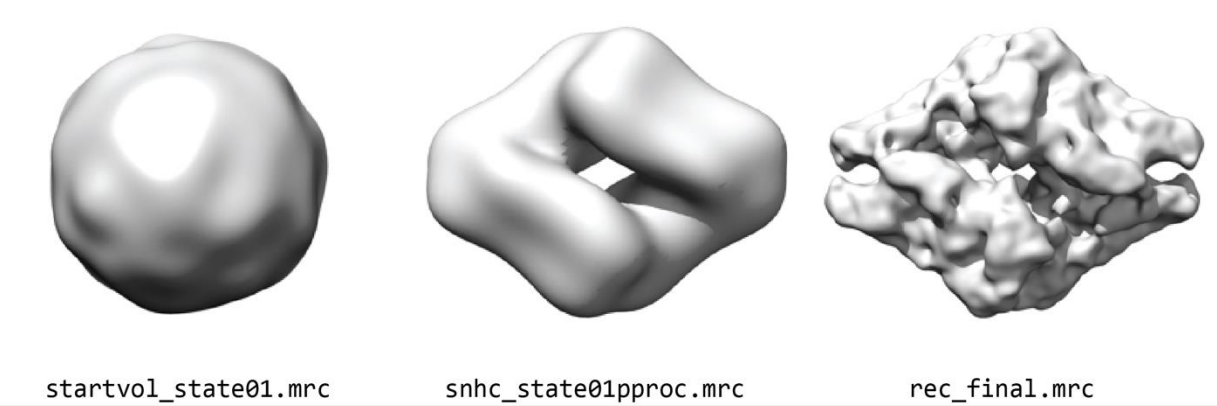
2 × 2 docking correlation matrix

Direct assignment	state 1 ↔ truth 1 state 2 ↔ truth 2
Swapped assignment	state 1 ↔ truth 2 state 2 ↔ truth 1

Accept the better one-to-one assignment, then require each matched correlation to clear the scientific floor.

State numbering remains arbitrary, but scientific identity does not.

Case study: numerical tests combine truth and controls



FIXED-SEED PHANTOM

0.3897 raw correlation

0.8153 enhanced correlation

A robust numerical test includes

- A deterministic asymmetric truth.
- An independent metric such as correlation or FSC.
- A negative or opposite control.
- Finite-value and lifecycle checks.
- A tolerance tied to the scientific claim.

The test judges whether the algorithm improves the truth metric, not whether it merely writes an output map.

SIMPLE runs each kind of evidence at the right cadence

Mechanism	Question	Cadence
Fast gate	Did the change break a local contract?	Every normal build, before install
Focused area suite	Did the touched subsystem remain correct?	During implementation and review
Nightly / library	Do realistic numerical fixtures still work?	Background or scheduled
High-level entry	Does the workflow recover the planted outcome?	Explicit, long-running validation
Profile / identity	Is the result robust and reproducible?	Across seeds, threads, hosts, or two run directories

The 60-second fast-gate budget is a warning threshold. A failed entry still fails the build and prevents installation.

The registry makes large-test ownership operational

ONE ROW DECLARES

- CTest name and simple_test_exec selector
- Threads, workers, CPU reservation, and deadline
- Memory and disk estimates
- Correctness gate or robustness profile
- Output patterns used by identity checks

Consistency checks

check_test_registry.py

holds CMake, UI selectors, routers, and the high-level registry to the same contract

CTest process budget

prevents suite growth from silently creating more isolation units

Recorded ownership and resource needs let nightly orchestration route work and failures predictably.

Automation cannot answer every cryo-EM question

AUTOMATION CAN PROVE

- A closed-form or planted-truth relationship
- A file and project lifecycle contract
- Thread or process identity within a tolerance
- A registered workflow's declared gate

PEOPLE STILL JUDGE

- Whether a map is biologically interpretable
- Whether a benchmark represents real user data
- Whether a metric measures the intended quality
- Whether a platform result has actually been observed

SIMPLE's rule is evidence-specific: never claim Linux or BOX passed unless that output was observed.

SIMPLE turns testing principles into scientific evidence

- A test fails through assertions, not through intention.
- Fast nearby tests localize defects before workflows become involved.
- Expected values come from independent mathematics, emulation, or planted truth.
- Fixed seeds replay failures. Profiles measure robustness.
- Larger tests prove integration and recovery with clear ownership.

Testing gives SIMPLE confidence to improve without mistaking completion for correctness.